SIMATS ENGINEERING
Department of Computer Science & Engineering
ASSESSMENT TOOL 2- COMPARATIVE ANALYSIS

Course Code:	**CSA12**	**Course Title:**	**Computer Architecture**
CO Assessed:	**CO4**	**Assessment:**	**ASSESSMENT TOOL2- COMPARATIVE ANALYSIS**
Weightage:	**30%**	**Total Marks:**	**25**
CO4 Statement:	*Implement hierarchical memory systems by differentiating cache levels (L1, L2, L3), virtual memory with paging, and advanced memory technologies (DRAM, SRAM, NAND Flash, MRAM, RRAM) based on latency, bandwidth, and throughput characteristics.*		
Student Name:	N.AKHIL KUMAR	**Reg.No.:**	192425128
Department:	AIML	**Date:**	

ANNEXURE A
COMPARATIVE ANALYSIS

1. Compare L1, L2, and L3 cache levels in terms of latency, bandwidth, and throughput, and analyze their impact on processor performance.
2. Differentiate SRAM and DRAM by evaluating their latency, bandwidth, and throughput characteristics in cache and main memory design.
3. Compare virtual memory with paging and cache memory mechanisms in terms of access latency and system throughput.
4. Analyze the differences between NAND Flash and DRAM with respect to latency, bandwidth, and suitability for hierarchical memory systems.
5. Compare MRAM and RRAM technologies by examining their performance in terms of access time, throughput, and energy efficiency.

Code of Conduct:

I N;AKHIL KUMAR(192425128) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: N.AKHIL KUMAR

MARKS ALLOCATION

	Scope of Items (20%)	Comparison Depth (25%)	Use of Evidence (20%)	Critical Thinking (20%)	Clarity of Presentation (15%)
Q1					
Q2					
Q3					
Q4					
Q5					

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Scope of Items	20%	Selects highly relevant items with clear scope and justification	Selects appropriate items with minor gaps	Selection is limited or partially relevant	Items are unclear or not relevant
Comparison Depth	25%	Provides detailed and comprehensive comparison across multiple aspects	Provides adequate comparison with some depth	Limited comparison with few aspects	Very minimal or no comparison
Use of Evidence	20%	Supports comparison with accurate data, examples, or references	Uses some supporting evidence with minor gaps	Limited or weak supporting evidence	No supporting evidence provided
Critical Thinking	20%	Demonstrates strong critical thinking with clear interpretation and reasoning	Shows reasonable interpretation with minor gaps	Basic interpretation; lacks depth	No meaningful interpretation
Clarity of Presentation	15%	Well-organized, clear, and logically structured presentation	Generally clear with minor issues	Somewhat unclear or poorly structured	Disorganized and difficult to understand

ANSWERS:

Q1. Comparison of L1, L2, and L3 Cache

Introduction

CPU cache is a small, high-speed memory located close to the processor. It stores frequently accessed instructions and data so that the CPU does not need to access slower main memory repeatedly. Modern processors generally use multiple cache levels: **L1, L2, and L3**.

Comparison

Feature	L1 Cache	L2 Cache	L3 Cache
Location	Closest to CPU core	Close to CPU core	Usually shared among cores
Capacity	Smallest	Medium	Largest
Access latency	Lowest	Low	Higher than L1/L2
Bandwidth	Very high	High	High
Throughput	Very high	High	High
Cost per bit	Highest	High	Lower than L1/L2
Typical use	Immediate instructions/data	Frequently reused data	Shared/recently used data
Sharing	Usually per core	Per core or cluster	Usually shared
Miss penalty	Lowest	Moderate	Higher
Main benefit	Minimum latency	Balance of speed and capacity	Reduces DRAM accesses

Latency

The typical relationship is:

L1 < L2 < L3 < DRAM

L1 is designed for extremely fast access because it is located closest to the processor. L2 provides more capacity at slightly higher latency. L3 is larger and often shared, but its access latency is higher.

Exact latency values depend heavily on the processor architecture, clock frequency, cache design, and workload, so fixed numerical values should not be treated as universal.

Bandwidth and Throughput

L1 generally provides the highest effective bandwidth to an individual CPU core because it is extremely close to the execution units. L2 provides substantial bandwidth with greater capacity, while L3 provides a larger shared data pool.

Cache **throughput** depends not only on cache size but also on:

- Number of cache banks.
- Number of ports.
- CPU frequency.
- Number of cores.
- Cache-line size.

- Memory access pattern.
- Hit rate.

Impact on Processor Performance

Consider a processor executing a program:

CPU Request

↓

L1 Cache

↓ Cache Hit

Data immediately available

If the data is not in L1:

CPU

↓

L1 Miss

↓

L2

↓

L3

↓

DRAM

Every additional level increases the time required to obtain the data.

Critical Analysis

Increasing cache size does not automatically guarantee better performance. A larger cache can improve the hit rate, but it can also increase complexity, area, and sometimes access latency. Therefore, processors use a **hierarchical design**:

L1 → very low latency

L2 → latency/capacity balance

L3 → large shared cache

This hierarchy allows the processor to achieve high performance without requiring the entire memory system to be built from expensive SRAM.

Conclusion

L1, L2, and L3 caches form a hierarchy in which **L1 prioritizes minimum latency, L2 balances latency and capacity, and L3 provides larger shared storage while reducing DRAM accesses**. Efficient cache hierarchy improves CPU throughput by increasing cache-hit rates and reducing processor stalls.

Q2. SRAM vs DRAM

Introduction

SRAM and DRAM are both volatile semiconductor memories, but their internal structures make them suitable for different levels of the memory hierarchy.

SRAM is primarily used for CPU caches, whereas **DRAM** is primarily used as main memory.

Comparison

Feature	SRAM	DRAM
Storage mechanism	Flip-flop cells	Capacitor-based cells
Latency	Very low	Higher
Bandwidth	Very high in cache applications	High
Throughput	Very high for small working sets	High for large data transfers
Density	Low	High
Cost per bit	High	Lower
Refresh	Not required	Required
Power	No refresh power, but larger cell area	Refresh required
Typical use	L1/L2/L3 cache	Main memory
Capacity	Small	Large

SRAM in Cache Design

SRAM is used for cache because the processor needs very fast access.

CPU

↓

SRAM Cache

↓

DRAM

The small physical size of SRAM cells is a disadvantage in terms of density, but its speed makes it ideal for L1, L2, and L3 caches.

DRAM in Main Memory

DRAM provides much greater capacity at lower cost per bit.

It is suitable for:

- Operating systems.
- Applications.
- Databases.
- Large datasets.
- AI workloads.

However, DRAM requires periodic refreshing and has greater access latency than SRAM.

Latency

The main distinction is:

SRAM → lower latency

DRAM → higher latency

When a CPU finds data in SRAM cache, it can continue execution much faster than if the data has to be retrieved from DRAM.

Bandwidth and Throughput

DRAM can provide extremely high aggregate bandwidth, particularly with multiple memory channels and modern DDR technologies. SRAM provides very high low-latency bandwidth at the cache level.

Therefore, saying simply that "SRAM always has higher bandwidth than DRAM" is not technically sufficient. **SRAM has the latency advantage, while modern DRAM can provide enormous aggregate bandwidth because of its high density and multiple channels.**

Critical Analysis

SRAM cannot simply replace DRAM because doing so would make large-memory systems extremely expensive and physically inefficient.

Similarly, using only DRAM for caches would increase processor access latency.

The optimal design is:

CPU
↓
SRAM Cache
↓
DRAM Main Memory
↓
SSD/Storage

Conclusion

SRAM is ideal for **high-speed cache memory**, while DRAM is ideal for **large-capacity main memory**. Their complementary characteristics allow computer systems to achieve both low latency and large memory capacity.

Q3. Virtual Memory, Paging, and Cache Memory

Introduction

Cache memory and virtual memory are different mechanisms that improve or manage memory access in different ways.

- **Cache memory** reduces CPU memory-access latency.
- **Virtual memory** provides an abstraction between virtual and physical addresses.
- **Paging** is a mechanism used by virtual-memory systems to manage memory in fixed-size pages.

Comparison

Feature	Cache Memory	Virtual Memory	Paging
Primary purpose	Reduce access latency	Provide virtual address space	Manage memory in pages
Managed mainly by	Hardware	OS + hardware	OS + hardware
Basic unit	Cache line	Virtual page	Page
Fast storage	SRAM cache	DRAM	DRAM + storage when necessary
Normal latency	Very low	Depends on translation/cache state	Can become very high on page fault
Storage access	Normally not required	May use storage as backing store	May require SSD/storage

Feature	Cache Memory	Virtual Memory	Paging
Effect on throughput	Usually improves	Improves flexibility/isolation	Excessive paging can reduce throughput

Cache Mechanism

When the CPU requests data:

CPU
↓
Cache
↓ Hit
Data

If the data is not present:

CPU
↓
Cache Miss
↓
Lower Cache / DRAM

A cache miss increases access time but does not normally require a disk/SSD operation.

Virtual Memory

Virtual memory gives processes a virtual address space.

Virtual Address
↓
Address Translation
↓
Physical Memory

Modern systems commonly use translation caches such as the **TLB (Translation Lookaside Buffer)** to reduce the cost of repeated address translations.

Paging

Paging divides memory into fixed-size pages.

If a required page is already in DRAM:

Virtual Page
↓
Physical Page in DRAM

If it is not available and has to be fetched from secondary storage:

Page Fault
↓
SSD/Storage
↓
DRAM
↓
CPU

This can be dramatically slower than a cache access.

Impact on Throughput

Cache memory generally improves throughput by reducing CPU stalls.

Paging can improve overall memory utilization, but **excessive page faults can severely reduce system throughput**. This condition is often called **thrashing** when the system spends excessive time moving pages instead of executing useful work.

Critical Analysis

Cache and paging should not be treated as competing technologies.

They operate at different levels:

CPU

↓

Cache

↓

DRAM

↓

Virtual-memory backing storage

A well-designed system needs both.

Conclusion

Cache memory primarily addresses **processor latency**, while virtual memory and paging primarily address **memory management, isolation, and capacity**. Cache misses generally have much smaller penalties than page faults involving storage. Therefore, sufficient DRAM and effective caching are essential for high system throughput.

Q4. NAND Flash vs DRAM

Introduction

NAND Flash and DRAM are both important memory technologies, but they serve very different purposes.

DRAM is volatile, high-speed working memory, while **NAND Flash** is non-volatile storage memory.

Comparison

Feature	DRAM	NAND Flash
Volatility	Volatile	Non-volatile
Latency	Very low compared with storage	Much higher
Bandwidth	High	High sequential bandwidth possible
Random access	Excellent	Much slower
Throughput	High for memory workloads	Good for storage workloads
Persistence	No	Yes
Refresh	Required	Not required
Typical use	Main memory	SSDs, phones, embedded storage
Write behavior	Fast memory writes	Program/erase operations are slower
Best role	Active working data	Persistent data

Latency

DRAM is designed for frequent random accesses by the CPU.

NAND Flash is designed primarily for persistent storage. Its access latency is much higher than DRAM, particularly for small random operations.
Therefore:
DRAM → low-latency working memory
NAND → persistent high-capacity storage
Bandwidth
Modern NAND-based SSDs can provide high sequential read/write throughput, especially when multiple NAND channels operate in parallel.
However, storage bandwidth should not be confused with DRAM latency.
For example, a storage device may achieve very high sequential throughput while still having much greater access latency than DRAM.
Hierarchical Memory Design
A typical hierarchy is:
CPU
↓
L1/L2/L3 Cache
↓
DRAM
↓
NAND Flash / SSD
DRAM
Used for:

- Running applications.
- Active datasets.
- CPU working memory.
- AI model execution.
- Database buffer pools.

NAND Flash
Used for:

- Operating-system storage.
- Applications.
- Files.
- Large datasets.
- AI model storage.

Critical Analysis
Replacing DRAM with NAND Flash would significantly reduce cost per bit and provide persistence, but active programs would experience much higher access latency.
Replacing NAND Flash with DRAM would provide faster access but would lose persistence and become much more expensive for large capacities.
Therefore, they are **complementary rather than interchangeable**.

Conclusion

DRAM should be positioned close to the processor as high-speed working memory, while NAND Flash should provide persistent high-capacity storage.

The hierarchy balances:
DRAM → performance
NAND → capacity and persistence

Q5. MRAM vs RRAM

Introduction

MRAM and RRAM are emerging/non-volatile memory technologies designed to provide persistent data storage with potentially better performance and energy characteristics than traditional storage technologies in selected applications.
Their actual performance varies significantly by implementation, memory generation, process technology, and device architecture.

Comparison

Feature	MRAM	RRAM
Full form	Magnetoresistive RAM	Resistive RAM
Storage principle	Magnetic state	Resistance state
Non-volatile	Yes	Yes
Access time	Fast	Fast to moderate depending on implementation
Read performance	High	High
Write performance	High	Technology-dependent
Throughput	High	Potentially high
Standby power	Very low	Very low
Endurance	Generally high	Device-dependent
Density potential	High	Very high potential
Main challenge	Cost/density trade-offs	Variability and endurance
Possible applications	Embedded memory, caches, persistent memory	Embedded memory, storage, compute-in-memory

MRAM
MRAM stores information using magnetic states.
A major advantage is that MRAM is **non-volatile**, meaning data remains stored when power is removed.
Advantages include:

- Fast read access.
- Good endurance.
- Low standby power.
- Non-volatility.
- Suitable for embedded applications.

MRAM can therefore be useful where both fast access and persistence are required.

RRAM

RRAM stores information by changing the resistance of a memory cell.

Potential advantages include:

- High density.
- Non-volatility.
- Low standby power.
- Potential for highly parallel architectures.
- Potential suitability for compute-in-memory AI applications.

However, practical RRAM implementations can face challenges involving:

- Device variability.
- Resistance-state distributions.
- Write reliability.
- Endurance.
- Manufacturing consistency.

Access Time

Both technologies can offer substantially faster access than conventional NAND storage in appropriate implementations.

However, exact access time should not be stated as a universal fixed value because it depends on the specific device architecture.

Throughput

MRAM can provide high read/write throughput for applications requiring fast persistent memory.

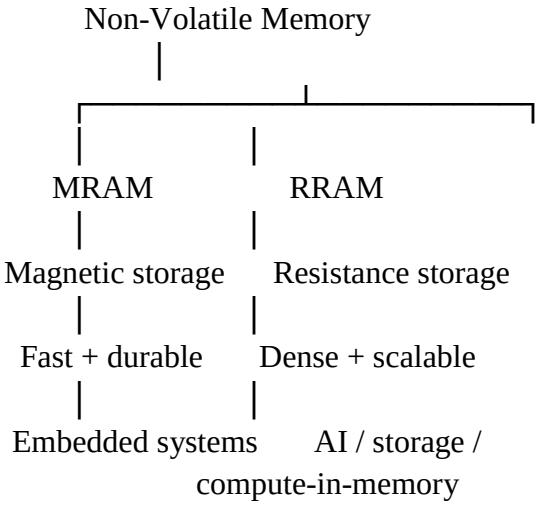
RRAM has significant potential for high parallel throughput, especially in specialized architectures.

For AI applications, RRAM is particularly interesting because its physical properties can potentially support **compute-in-memory**, reducing data movement between processor and memory.

Energy Efficiency

Both technologies can reduce standby energy because they are non-volatile.

A simplified comparison is:



Critical Analysis

MRAM and RRAM should not be judged only by access time.
A complete evaluation should consider:

- Read latency.
- Write latency.
- Endurance.
- Energy per operation.
- Density.
- Manufacturing cost.
- Data retention.
- Variability.
- Application requirements.

For applications requiring high endurance and predictable fast access, MRAM can be attractive. For highly dense architectures and specialized AI/compute-in-memory applications, RRAM can be attractive.

Conclusion

MRAM provides a strong combination of **speed, endurance, low standby power, and non-volatility**, while RRAM offers strong potential in **density, low-power operation, and compute-in-memory architectures**. The best choice depends on the workload and implementation rather than one technology being universally superior.